

Roadmap — smart-flight-app

Titre Professionnel Développeur en Intelligence Artificielle — Bloc 3 (C14 → C21)

Concept du projet

Problème adressé

Un voyageur qui veut estimer le coût d'un billet d'avion doit naviguer entre plusieurs plateformes sans avoir de vue claire sur ce qui influence réellement le prix. Il n'existe pas d'outil simple qui lui permette d'entrer ses paramètres de voyage et d'obtenir immédiatement une estimation de prix, avec la conversion dans sa devise locale.

Solution

smart-flight-app est une application web construite avec Streamlit. L'utilisateur entre les caractéristiques de son vol (compagnie, trajet, classe, escales, durée, jours avant le départ), l'application interroge l'API de prédiction du Bloc 2 (**smart-flight-pricer**) et affiche le prix estimé en INR avec la conversion automatique en euros via un taux de change en temps réel.

Lien avec le Bloc 2

L'application consomme directement l'endpoint (**POST /predict**) de l'API FastAPI développée en Bloc 2. Elle ne contient aucun modèle ML en propre — elle est le **client applicatif** du service IA du Bloc 2.

Architecture globale

```
[Utilisateur – navigateur web]
  ↓ formulaire Streamlit
[smart-flight-app – Streamlit (port 8501)]
  ↓ POST /predict + X-API-Key
[smart-flight-pricer – API FastAPI (port 8000)]
  ↓ prédiction RandomForest
[Prix en INR]
  ↓ GET taux INR/EUR
[frankfurter.app – API taux de change]
  ↓ conversion
[Affichage : "7 838 INR – soit ~86 €"]
  ↓ log applicatif
[Monitoring applicatif – journalisation + alertes]
```

Compétences couvertes

Compétence	Description courte
C14	Spécifications fonctionnelles, user stories, modélisation parcours utilisateur
C15	Architecture technique, stack, diagramme de flux de données, preuve de concept
C16	Méthode agile — kanban, backlog, rituels documentés
C17	Développement Streamlit, intégration API Bloc 2, sécurité OWASP, tests
C18	CI/CD — GitHub Actions sur l'application Streamlit
C19	Processus de livraison continue — packaging, déploiement
C20	Monitoring applicatif — logs, métriques, alertes
C21	Résolution d'incident — identification, correction, documentation

📁 Structure du projet

```
smart-flight-app/
|
├── .github/
|   └── workflows/
|       └── ci.yml                # Pipeline CI/CD GitHub Actions
|
├── app/
|   ├── main.py                  # Application Streamlit principale
|   ├── api_client.py            # Client HTTP vers l'API Bloc 2
|   └── currency.py              # Conversion INR → EUR via
frankfurter.app
|
├── monitoring/
|   └── logger.py                # Journalisation des requêtes
applicatives
|   └── logs/                    # Logs JSONL (non versionné)
|
├── tests/
|   ├── conftest.py              # Fixtures partagées pytest
|   ├── test_api_client.py       # Tests du client HTTP
|   └── test_currency.py         # Tests de la conversion de devise
|
└── docs/
```

```
|   ├── specs_fonctionnelles.md      # C14 – User stories + parcours
utilisateur
|   ├── specs_techniques.md          # C15 – Architecture + diagramme flux
|   ├── agile.md                     # C16 – Backlog + kanban + rituels
|   ├── monitoring.md                # C20 – Documentation du monitoring
|   └── incident.md                  # C21 – Rapport d'incident documenté
|
└── .env                             # Variables d'environnement (non
versionné)
└── .gitignore
└── requirements.txt
└── README.md
```

Phase 0 — Initialisation (Jour 1)

Durée : 15 minutes **Compétences :** aucune — mise en place

Actions

- Créer le repo Git (`smart-flight-app`)
- Créer la structure de dossiers
- Créer le `.gitignore` (venv, .env, logs)
- Créer le `requirements.txt` de base : `streamlit`, `requests`, `pytest`, `python-dotenv`
- Premier commit + push

Phase 1 — Spécifications (C14, C15, C16)

Durée : 2 jours **Compétences :** C14, C15, C16

C14 — Spécifications fonctionnelles

Fichier : `docs/specs_fonctionnelles.md`

Contenu attendu

- Contexte et objectifs du projet
- Acteurs : utilisateur final (voyageur français)
- Parcours utilisateur : schéma étape par étape (entrée formulaire → appel API → affichage résultat)
- User stories au format : *"En tant que voyageur, je veux entrer mon trajet afin d'obtenir une estimation de prix"*
- Critères d'acceptation pour chaque user story
- Objectifs d'accessibilité (WCAG — contrastes, labels formulaires)

User stories minimales

ID	User Story	Critère d'acceptation
US1	En tant qu'utilisateur, je veux sélectionner ma compagnie, mon trajet et ma classe	Le formulaire affiche les valeurs disponibles en liste déroulante
US2	En tant qu'utilisateur, je veux obtenir un prix estimé en INR et en euros	L'application affiche les deux devises après soumission
US3	En tant qu'utilisateur, je veux être informé si l'API est indisponible	Un message d'erreur clair s'affiche sans crash de l'application

C15 — Spécifications techniques

Fichier: `docs/specs_techniques.md`

Contenu attendu

- Architecture n-tiers : présentation (Streamlit) → service IA (FastAPI Bloc 2) → données externes (frankfurter.app)
- Stack technique avec justifications
- Diagramme de flux de données (DFD) — texte ou schéma ASCII
- Variables d'environnement nécessaires
- Preuve de concept : tester que l'appel API Bloc 2 fonctionne avant de développer l'interface

Stack technique

Couche	Technologie	Justification
Interface	Streamlit	Rapide, Python pur, pas de CSS requis
Client HTTP	requests	Standard Python, simple
Conversion devise	frankfurter.app	Gratuit, sans clé API, taux en temps réel
Tests	pytest	Cohérence avec Bloc 2
CI/CD	GitHub Actions	Cohérence avec Bloc 2
Logs	JSONL	Léger, lisible, cohérent avec Bloc 2

Diagramme de flux de données



C16 — Méthode agile

Fichier : `docs/agile.md`

Contenu attendu

- Méthode retenue : Scrum simplifié (adapté à un projet solo)
- Backlog priorisé des tâches
- Tableau kanban : To Do / In Progress / Done
- Rituels documentés : daily solo (5 min/jour de note d'avancement), sprint review à mi-projet

Backlog minimum

Priorité	Tâche	Statut
1	Initialisation repo + structure	✓ Done
2	Spécifications fonctionnelles et techniques	✓ Done
3	Preuve de concept — appel API Bloc 2	To Do
4	Interface Streamlit — formulaire	To Do
5	Intégration appel API + affichage résultat INR	To Do
6	Conversion EUR via frankfurter.app	To Do
7	Gestion des erreurs (API down, timeout)	To Do
8	Journalisation des requêtes	To Do
9	Tests automatisés	To Do
10	CI/CD GitHub Actions	To Do
11	Documentation monitoring	To Do
12	Incident documenté (C21)	To Do
13	README complet	To Do

Phase 2 — Interface Streamlit (C17)

Durée : 2-3 jours **Compétence :** C17

Ce que l'interface doit faire

1. Afficher un formulaire avec les champs suivants :

- Compagnie aérienne (selectbox)
- Ville de départ (selectbox)
- Ville d'arrivée (selectbox)
- Nombre d'escales (selectbox)
- Classe (selectbox — Economy / Business)
- Durée du vol en heures (number_input)
- Jours avant le départ (slider 1-365)

2. Bouton "Estimer le prix"

3. Afficher le résultat : "Prix estimé : 7 838 INR — soit ~86 €"

4. Afficher un message d'erreur clair si l'API est indisponible

Structure de `app/main.py`

python

```
import streamlit as st
from app.api_client import predict_price
from app.currency import convert_inr_to_eur

st.title("✈️ Smart Flight Pricer")
st.caption("Estimation du prix d'un billet basée sur l'IA")

# Formulaire
airline = st.selectbox("Compagnie", ["Air_India", "Vistara", "Indigo", "GO_FIRS"])
source_city = st.selectbox("Ville de départ", ["Delhi", "Mumbai", "Bangalore"])
destination_city = st.selectbox("Ville d'arrivée", ["Delhi", "Mumbai", "Bangalore"])
stops = st.selectbox("Escales", ["zero", "one", "two_or_more"])
flight_class = st.selectbox("Classe", ["Economy", "Business"])
duration = st.number_input("Durée (heures)", min_value=0.5, max_value=50.0, value=1.0)
days_left = st.slider("Jours avant le départ", 1, 365, 30)

if st.button("Estimer le prix"):
    result = predict_price(airline, source_city, destination_city, stops, flight_class, duration, days_left)
    if result:
        prix_inr = result["predicted_price"]
        prix_eur = convert_inr_to_eur(prix_inr)
        st.success(f"Prix estimé : **{prix_inr:,.0f} INR** – soit **~{prix_eur:,.0f} EUR**")
    else:
        st.error("Le service de prédiction est actuellement indisponible. Veuillez réessayer plus tard.")
```

Structure de `app/api_client.py`

python

```
import requests
import os
from dotenv import load_dotenv

load_dotenv()
API_URL = os.getenv("API_URL", "http://127.0.0.1:8000")
API_KEY = os.getenv("API_KEY")

def predict_price(airline, source_city, destination_city, stops, flight_class,
    try:
        response = requests.post(
            f"{API_URL}/predict",
            headers={"X-API-Key": API_KEY},
            json={
                "airline": airline,
                "source_city": source_city,
                "destination_city": destination_city,
                "stops": stops,
                "class": flight_class,
                "duration": duration,
                "days_left": days_left
            },
            timeout=5
        )
        response.raise_for_status()
        return response.json()
    except Exception:
        return None
```

Structure de `app/currency.py`

python


```
import requests

def convert_inr_to_eur(amount_inr: float) -> float:
    try:
        response = requests.get(
            "https://api.frankfurter.app/latest?from=INR&to=EUR",
            timeout=3
        )
        taux = response.json()["rates"]["EUR"]
        return round(amount_inr * taux, 2)
    except Exception:
        # Fallback taux fixe si l'API est indisponible
        return round(amount_inr * 0.011, 2)
```

Variables d'environnement `.env`

```
env

API_URL=http://127.0.0.1:8000
API_KEY=votre_cle_secrete_bloc2
```

Critères jury (C17)

- Interface fonctionnelle et conforme aux spécifications
- Communication avec l'API Bloc 2 opérationnelle
- Gestion des erreurs — l'application ne crashe pas si l'API est down
- Authentification gérée côté client (X-API-Key)
- Sources versionnées sur Git

Livrables

- `app/main.py`
- `app/api_client.py`
- `app/currency.py`

Phase 3 — Journalisation et monitoring (C20)

Durée : 1-2 jours **Compétence :** C20

Ce qu'il faut monitorer (niveau applicatif)

Métrique	Seuil d'alerte	Description
Nombre de requêtes par heure	> 100/h	Détection de pic inhabituel
Taux d'erreurs API Bloc 2	> 10%	L'API Bloc 2 devient instable
Latence de réponse	> 3s	L'expérience utilisateur se dégrade
Indisponibilité API change	> 3 échecs consécutifs	Fallback taux fixe activé

Structure de `monitoring/logger.py`

```
python

import json
import os
from datetime import datetime

LOG_DIR = os.getenv("LOG_DIR", "monitoring/logs")
LOG_FILE = os.path.join(LOG_DIR, "app.jsonl")

def log_request(inputs: dict, result: dict | None, latency_ms: float, error: str):
    os.makedirs(LOG_DIR, exist_ok=True)
    entry = {
        "timestamp": datetime.utcnow().isoformat(),
        "inputs": inputs,
        "result": result,
        "latency_ms": round(latency_ms, 2),
        "error": error
    }
    with open(LOG_FILE, "a") as f:
        f.write(json.dumps(entry) + "\n")
```

Critères jury (C20)

- Métriques définies et justifiées
- Journalisation intégrée dans l'application
- Alertes configurées (au minimum documentées)
- Documentation du monitoring rédigée
- Sources versionnées sur Git

Livrables

- `monitoring/logger.py`
 - `docs/monitoring.md`
-

Phase 4 — Tests automatisés (C17, C18)

Durée : 1-2 jours **Compétences :** C17, C18

Cas de tests à couvrir

- `test_api_client.py` : réponse correcte quand l'API répond, retour `None` quand l'API est down (mock), timeout géré
- `test_currency.py` : conversion INR → EUR retourne un float positif, fallback taux fixe si frankfurter.app est indisponible

Commande

```
bash

pytest tests/ --cov=app --cov-report=term
```

Livrables

- `tests/test_api_client.py`
 - `tests/test_currency.py`
 - `tests/conftest.py`
-

Phase 5 — CI/CD (C18, C19)

Durée : 1 jour **Compétences :** C18, C19

`.github/workflows/ci.yml`

```
yaml
```

```
name: CI - smart-flight-app
on:
  push:
    branches: [main, dev]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with:
          python-version: '3.12'
      - run: pip install -r requirements.txt
      - run: pytest tests/ --cov=app --cov-report=term
    env:
      API_URL: http://localhost:8000
      API_KEY: test-key
```

Critères jury (C18, C19)

- Déclenchement sur push et pull request
- Tests s'exécutent automatiquement
- Configuration versionnée sur Git
- Documentation de la chaîne rédigée

Livrables

- `.github/workflows/ci.yml`
- Badge CI dans `README.md`

Phase 6 — Incident technique (C21)

Durée : 1 jour **Compétence :** C21

Ce qu'il faut faire

Identifier un vrai bug dans ton application, le reproduire, le corriger et le documenter. Pas besoin d'inventer un problème — il y en aura forcément un pendant le développement.

Exemples d'incidents typiques à documenter

- L'application crashe si `source_city == destination_city`

- La conversion EUR retourne une valeur négative si le timeout est mal géré
- Le logger plante si le dossier `monitoring/logs/` n'existe pas

Structure de `docs/incident.md`

markdown

Rapport d'incident – [titre du bug]

Date

YYYY-MM-DD

Description

[Ce qui s'est passé]

Cause identifiée

[Pourquoi ça s'est produit]

Reproduction

[Étapes pour reproduire le bug]

Solution implémentée

[Ce qui a été corrigé + extrait de code]

Commit de correction

[Lien vers le commit Git]

Tests ajoutés

[Tests qui vérifient que le bug ne reviendra pas]

Critères jury (C21)

- Cause identifiée correctement
- Bug reproduit en environnement de développement
- Solution documentée étape par étape
- Correction versionnée dans le dépôt Git

Livrables

- `docs/incident.md`
 - Commit de correction sur Git avec message explicite
-

Phase 7 — Documentation et README (Semaine finale)

Durée : 1 jour

README doit contenir

- Description du projet et lien avec le Bloc 2
- Instructions d'installation en 3 commandes
- Comment lancer l'application : `streamlit run app/main.py`
- Comment lancer les tests
- Variables d'environnement nécessaires
- Tableau des compétences couvertes C14 → C21

Livrables

- `README.md` complet
- Rapport professionnel PDF
- Démo fonctionnelle prête

Vue calendrier réaliste

Jour	Focus	Compétences
Jour 1	Init + spécifications	C14, C15, C16
Jour 2	Interface Streamlit + appel API Bloc 2	C17
Jour 3	Conversion EUR + gestion erreurs + logging	C17, C20
Jour 4	Tests automatisés + CI/CD	C18, C19
Jour 5	Incident documenté + README + rapport	C21
Jour 6	Buffer — polish + vérification checklist	Tout

Checklist finale avant soutenance

E4 (C14, C15, C16, C17, C18, C19)

- ☐ User stories rédigées avec critères d'acceptation
- ☐ Architecture documentée avec diagramme de flux
- ☐ Backlog et kanban disponibles

- ☐ Application Streamlit fonctionnelle (formulaire + résultat)
- ☐ Appel API Bloc 2 opérationnel (`/predict`)
- ☐ Conversion INR → EUR affichée correctement
- ☐ Gestion des erreurs — pas de crash si API down
- ☐ Tests automatisés passent sans erreur
- ☐ Pipeline CI/CD se déclenche sur un push
- ☐ Sources versionnées sur Git

E5 (C20, C21)

- ☐ Journalisation des requêtes en JSONL
 - ☐ Métriques définies et documentées
 - ☐ Documentation monitoring rédigée
 - ☐ Incident identifié, reproduit et corrigé
 - ☐ Rapport d'incident rédigé dans `docs/incident.md`
 - ☐ Commit de correction versionné sur Git
-

Ce que le jury vérifiera à la démo

1. Tu lances `streamlit run app/main.py` → l'interface s'affiche
 2. Tu remplis le formulaire → tu obtiens un prix en INR et en euros
 3. Tu coupes l'API Bloc 2 → l'application affiche un message d'erreur sans crasher
 4. Tu montres `monitoring/logs/app.jsonl` → les requêtes sont loguées
 5. Tu lances `pytest` → tous les tests passent en vert
 6. Tu montres le pipeline CI/CD → il est passé au dernier push
 7. Tu présentes `docs/incident.md` → tu expliques le bug et comment tu l'as résolu
-

Dépendance critique

L'API du Bloc 2 doit tourner en local pour que l'application fonctionne.

Avant chaque démonstration, vérifier que `smart-flight-pricer` est lancé :

```
bash
cd ../smart-flight-pricer
uvicorn api.main:app --reload
```

Puis lancer l'application Bloc 3 :

```
bash
```

```
cd ../smart-flight-app  
streamlit run app/main.py
```

Roadmap générée le 12/06/2026 — À mettre à jour au fur et à mesure de l'avancement